

Tribhuvan University

BSc CSIT 7th Semester

Data Warehousing and Data Mining (CSC410)

Unit 3:

Data Preprocessing

By: Dabbal Singh Mahara

Amrit Campus

Data Quality

- **Data quality** refers to the fitness of data to serve its intended purpose in a given context, such as decision-making, analysis, operations, or planning.
- High-quality data leads to better decisions, while poor-quality data can result in errors, inefficiencies, and incorrect conclusions.

Factors of Data Quality

1. Accuracy

- Data correctly represents the real-world values or events it describes.
- Example: A customer's age recorded as 30 should truly be 30.

2. Completeness

- All required data is present.
- Example: A form requiring name, email, and phone number should have all fields filled.

3. Consistency

- Data is the same across multiple systems or datasets.
- Example: A customer's address should be the same in the billing and shipping databases.

4. Timeliness

- Data is up to date and available when needed.
- Example: Stock prices used in trading algorithms must reflect real-time values.

Factors of Data Quality

5. **Validity**

- Data conforms to the defined format, rules, or constraints.
- Example: A date of birth should be in a valid date format and within a reasonable range.

6. **Uniqueness**

- There are no duplicate records or data points.
- Example: A database should not have two entries for the same customer with the same ID.

7. **Relevance**

- Data is appropriate for the task or decision at hand.
- Example: Historical sales data is relevant for forecasting future sales.

Data Pre-processing

- Data preprocessing is required to handle these problems about real world data
- Thus, analysing these real world data without pre-processing is difficult and if somehow performed its results are not accurate, reliable, and effective.
- **Data pre-processing is done to improve the quality of Data and consequently the mining result.**
- **It is used to improve the efficiency and ease of the mining process.**

Major Tasks in Data Preprocessing

1. Data cleaning
2. Data integration
3. Data reduction
4. Data transformation
5. Data discretization

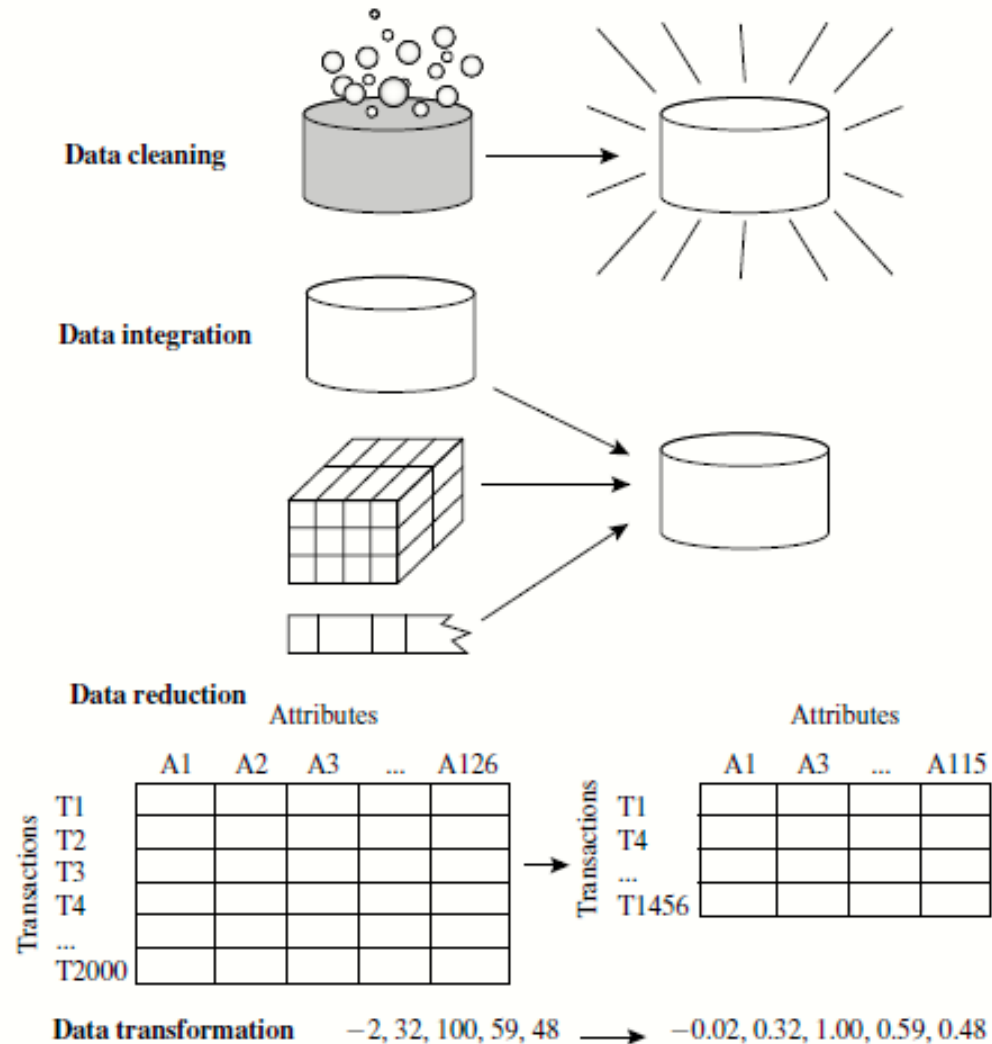


Fig. Data Preprocessing

Data Cleaning

- Data in the real world is dirty due to their typically huge size and possible origin from multiple, heterogeneous sources.
- In other words, the data we wish to analyze by data mining techniques are
 - **Incomplete:** lacking attribute values or certain attributes of interest, or containing only aggregate data.
 - **Noisy:** containing errors, or outlier values that deviate from the expected.
 - **Inconsistent:** e.g., containing discrepancies in the department codes used to categorize items.

Data Cleaning

- Data cleaning routines work to **“clean” the data by filling in missing values, smoothing noisy data, identifying or removing outliers, and resolving inconsistencies.**
- If users believe the data are dirty, they are unlikely to trust the results of any data mining that has been applied to it.
- Therefore, a useful preprocessing step is to run the data through some data cleaning routines.

Why can Data be Incomplete?

- Attributes of interest are not available (e.g., customer information for sales transaction data)
- Data were not considered important at the time of transactions, so they were not recorded!
- Data not recorder because of misunderstanding or malfunctions
- Data may have been recorded and later deleted!
- Missing/unknown values for some data

Why can Data be Noisy/Inconsistent?

- Faulty instruments for data collection
- Human or computer errors
- Errors in data transmission
- Technology limitations (e.g., sensor data come at a faster rate than they can be processed)
- Inconsistencies in naming conventions or data codes (e.g., 2/5/2002 could be 2 May 2002 or 5 Feb 2002)
- Duplicate tuples, which were received twice should also be removed

Handling missing values

- **Ignore the tuple:** usually done when class label is missing (assuming the tasks in classification)—not effective when the percentage of missing values per attribute varies considerably.
- **Fill in the missing value manually:** This approach is time-consuming and may not be feasible given a large data set with many missing values.
- **Use a global constant** to fill in the missing value: e.g., “unknown” or $-\infty$
- **Use the attribute mean** to fill in the missing value
- **Use the most probable value** to fill in the missing value: inference-based such as Bayesian formula or decision tree

Handling Noisy Data

- Noise is a random error or variance in a measured variable.
- Following data smoothing techniques are used to remove noise from data:
- **Binning**
 - first sort data and partition into (equal-frequency) bins
 - then one can smooth by bin means, smooth by bin median, smooth by bin boundaries, etc.
- **Regression**
 - smooth by fitting the data into regression functions
- **Clustering:**
 - detect and remove outliers
- **Combined computer and human inspection**
 - detect suspicious values and check by human (e.g., deal with possible outliers)

Binning

- Binning methods smooth a sorted data value by consulting its “neighbourhood,” i.e., the values around it.
- The sorted values are distributed into a number equal-frequency “buckets,” or *bins*.
- Then, one of the following binning techniques is used for smoothing:
 - **Smoothing by bin means :**
Each value in a bin is replaced by the mean value of the bin.
 - **Smoothing by bin medians:**
Each bin value is replaced by the bin median
 - **Smoothing by bin boundaries:**
Minimum and maximum values in a given bin are identified as the *bin boundaries*.
Each bin value is then replaced by the closest boundary value.

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

Smoothing by bin boundaries:

Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

Fig. Binning Methods for Data Smoothing

Data integration

- *Data integration is merging data* from multiple sources into a coherent data store, as in data warehousing.
- These sources may include multiple databases, data cubes, or flat files.
- There are a number of issues to consider during data integration.
 - **Schema integration and object matching**
 - **Redundancy is another important issue.**
 - **Detection and resolution of data value conflicts**
- Careful integration of the data from multiple sources can help reduce and avoid redundancies and inconsistencies in the resulting data set.
- This can help improve the accuracy and speed of the subsequent mining process.

Data Reduction

- Data reduction techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data.
- That is, mining on the reduced data set should be more efficient yet produce the same (or almost the same) analytical results.
- Data reduction strategies include:
 - **Dimensionality reduction:** It is the process of reducing the number of attributes under consideration. Wavelet transforms, principal component analysis and attribute subset selection methods are used.
 - **Numerosity reduction:** It is the process of replacing the original data volume by smaller forms of data representation. Regression, histograms, clustering, sampling and data cube aggregation are used for this purpose.
 - **Data Compression:** Applied to obtain reduced or compressed form of original data. Data compression can be lossy or lossless.

Data transformation

- In data transformation, the data are transformed or combined into forms appropriate for mining.
- Data transformation can involve the following:
 - i. **Smoothing:** works to remove noise from the data. Such techniques include binning, regression, and clustering.
 - ii. **Aggregation:** summary or aggregation operations are applied to the data. For example, the daily sales data may be aggregated so as to compute monthly and annual total amounts.
 - iii. **Discretization:** raw values of a numeric attributes are replaced by conceptual labels through the use of concept hierarchies. For example, values for numerical attributes, like age, may be mapped to higher-level concepts, like youth, middle-aged, and senior.
 - iv. **Normalization:** the attribute data are scaled so as to fall within a small specified range, such as -1.0 to 1.0 or 0.0 to 1.0.

Data Transformation by Normalization

- Normalization attempts to give all the attributes an equal weights.
- It is normally useful for classification and clustering algorithms.

a. Min-max normalization:

Suppose that $\min_A$ and $\max_A$ are the minimum and maximum values of an attribute, A Min-max normalization maps a value, v , of A to v' in the range $[\text{new_min}_A, \text{new_max}_A]$ by computing

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Eg. Let the income range is 12000 to 98000 be normalized to $[0.0, 1.0]$. Then 73600 is normalized to:

$$\frac{73600 - 12000}{98000 - 12000} (1.0 - 0) + 0 = 0.716$$

Data Transformation

b. z-score normalization:

In z-score normalization (or zero-mean normalization), the values for an attribute, A , are normalized based on the mean and standard deviation of A . A value, v , of A is normalized to v' by computing

$$v' = \frac{v - \text{mean}_A}{\sigma_A}$$

Eg. Let $\text{mean} = 54000$ and $\text{Standard Deviation } (\sigma) = 16000$. Then 73600 is mapped to

$$\frac{73600 - 54000}{16000} = 1.225$$

Data Transformation

c. Normalization by decimal scaling:

It normalizes the value by moving the decimal point of values of attribute A. The number of decimal points moved depends on the maximum absolute value of A. A value, v, of A is normalized to v' by computing

$$v' = \frac{v}{10^j}$$

Where j is the smallest integer such that $\text{Max}(|v'|) < 1$.

Example:

Suppose that the recorded values of A ranges from -986 to 917. The maximum absolute value of A is 986. To normalize by decimal scaling, we therefore divide each value by 1,000 (i.e., $j = 3$) so that -986 normalizes to -0.986 and 917 normalizes to 0.917.

Data Discretization and Concept Hierarchy Generation

- Discretization is a data preprocessing technique that converts continuous numerical data into a finite set of discrete values or intervals.
- Data discretization converts a large number of data values into smaller once, so that data evaluation and data management becomes very easy.
- Data discretization is a form of numerosity reduction that is very useful for the automatic generation of concept hierarchies.
- Example: We have an attribute of age with the following values.

Age	10,11,13,14,17,19,30, 31, 32, 38, 40, 42,70 , 72, 73, 75
-----	--

Table: Before discretization

Attribute	Age	Age	Age
	10,11,13,14,17,19,	30, 31, 32, 38, 40, 42	70 , 72, 73, 75
After Discretization	Young	Mature	Old

Data Discretization and Concept Hierarchy Generation

- Concept hierarchy organizes data attributes into multiple levels of abstraction, from more specific (low-level) concepts to more general (high-level) concepts.
- Raw data values for attributes are replaced by ranges or higher conceptual levels.
- Discretization and concept hierarchy generation are powerful tools for data mining, in that they allow the mining of data at multiple levels of abstraction.
- Data discretization and concept hierarchy generation are also forms of data reduction.
- This simplifies the original data and makes the mining more efficient. The resulting patterns mined are typically easier to understand.
- Concept hierarchies are also useful for mining at multiple abstraction levels.

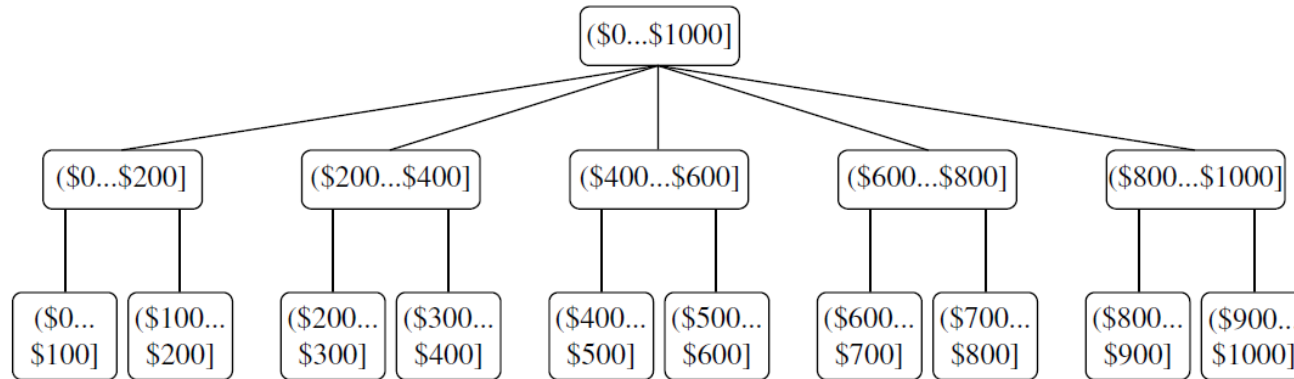
Concept hierarchy generation based on the number of distinct values per attribute.

- Consider the observation that since higher-level concepts generally cover several subordinate lower-level concepts, an attribute defining a high concept level (e.g., *country*) will usually contain a smaller number of distinct values than an attribute defining a lower concept level (e.g., *street*).
- Based on this observation, a concept hierarchy can be automatically generated based on the number of distinct values per attribute in the given attribute set.
- The attribute with the most distinct values is placed at the lowest hierarchy level.
- The lower the number of distinct values an attribute has, the higher it is in the generated concept hierarchy.

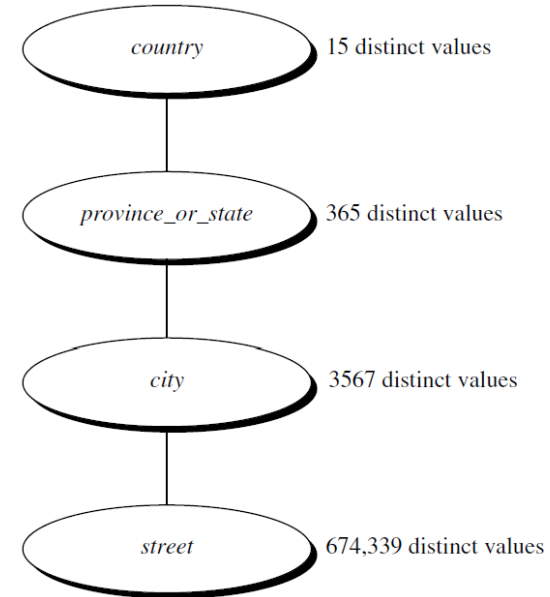
Concept hierarchy generation

- A concept hierarchy for *location* can be generated automatically, as illustrated in Figure.
- First, sort the attributes in ascending order based on the number of distinct values in each attribute.
- This results in the following (where the number of distinct values per attribute is shown in parentheses): *country* (15), *province or state* (365), *city* (3567), and *street* (674,339).
- Second, generate the hierarchy from the top down according to the sorted order, with the first attribute at the top level and the last attribute at the bottom level.

Data Discretization and Concept Hierarchy Generation



A concept hierarchy for the attribute *price*, where an interval $(\$X \dots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive).



Automatic generation of a schema concept hierarchy based on the number of distinct attribute values.

Data Mining Primitives

- Data mining primitives are the basic building blocks used to define a specific data mining task.
- Rather than expecting a system to autonomously discover all patterns (which would result in a "monster" of irrelevant information), these primitives allow users to interactively communicate with the system.
- They act as inputs to a Data Mining Query Language (DMQL) to guide the discovery process.

Primitives of a Data Mining Task

- **Task-Relevant Data:** Specifies the "What." Defines the specific portion of the database, tables, and relevant attributes to be analyzed.
- **Kind of Knowledge to be Mined:** Specifies the "Goal." Defines the function, such as Classification, Clustering, or Association.
- **Background Knowledge:** Provides "Context." Uses Concept Hierarchies to allow mining at different levels of abstraction (e.g., *Street* → *City* → *Country*).
- **Interestingness Measures:** Acts as a "Filter." Sets thresholds like Support and Confidence to separate useful knowledge from uninteresting patterns.
- **Knowledge Visualization:** Defines the "Output." Specifies how discovered patterns are presented (e.g., Rules, Tables, or Charts).

Thank You !